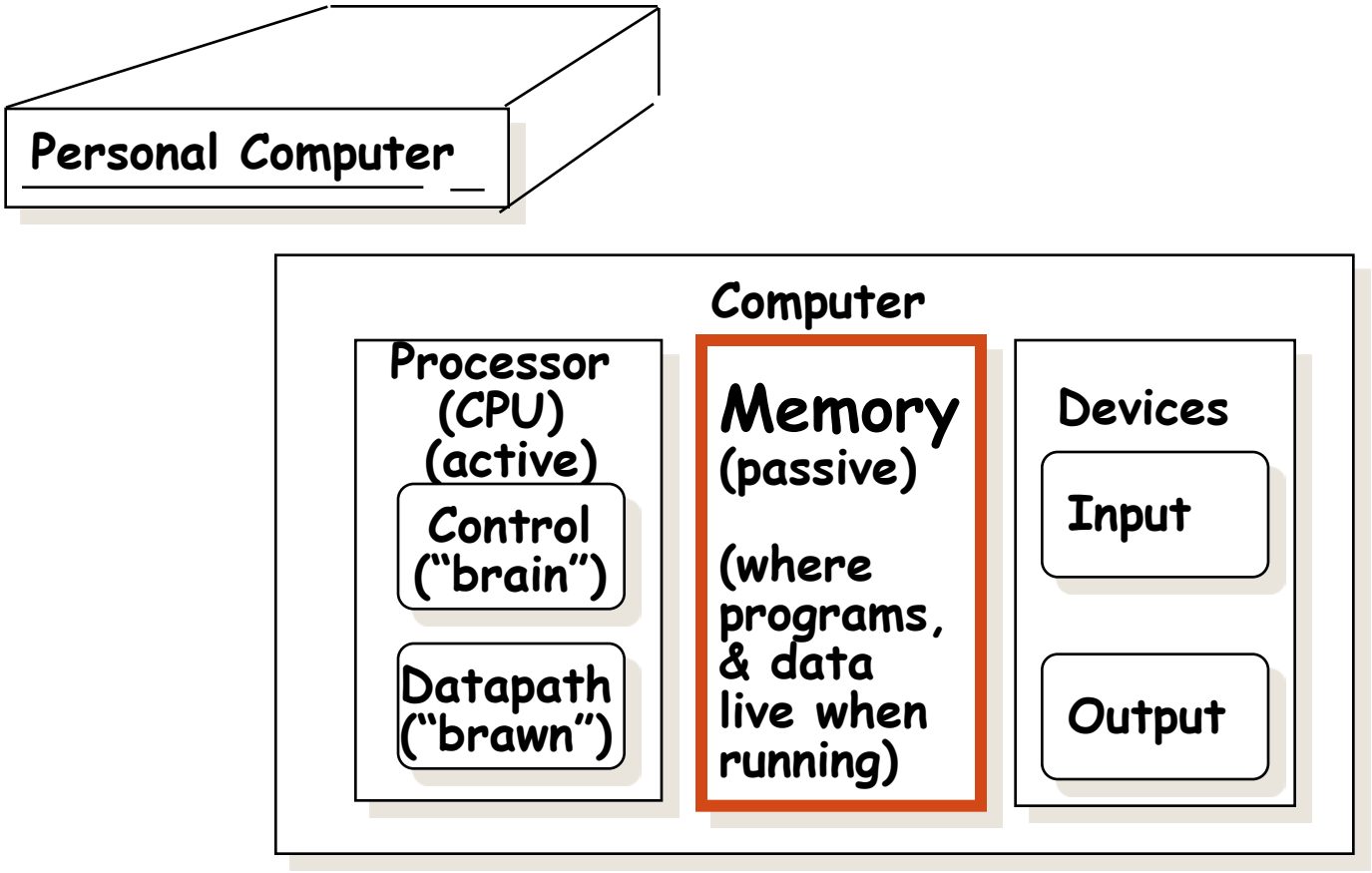
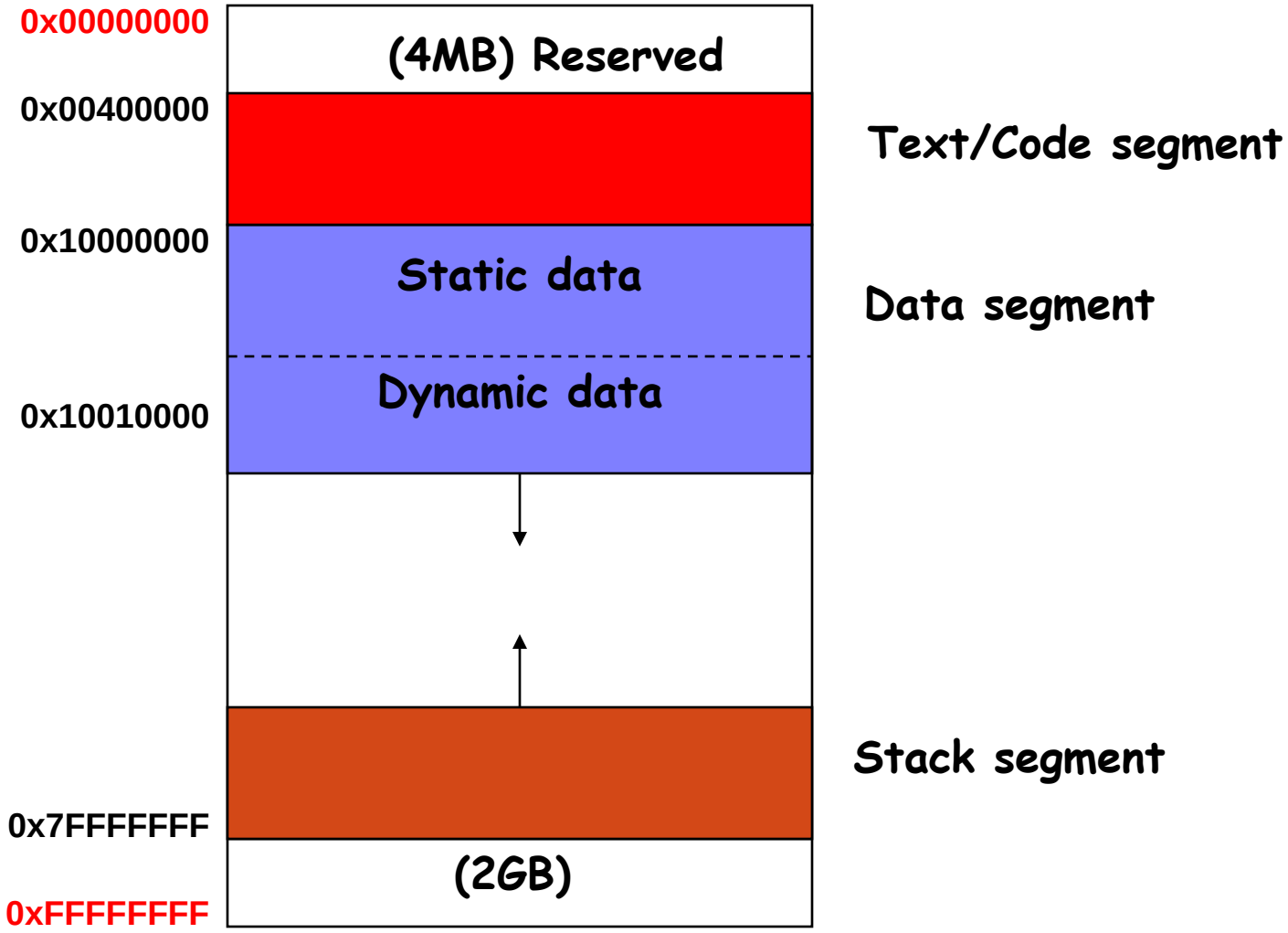
Memory Hierarchy
Virtual Memory and Cache

Recap: Computer Organization:

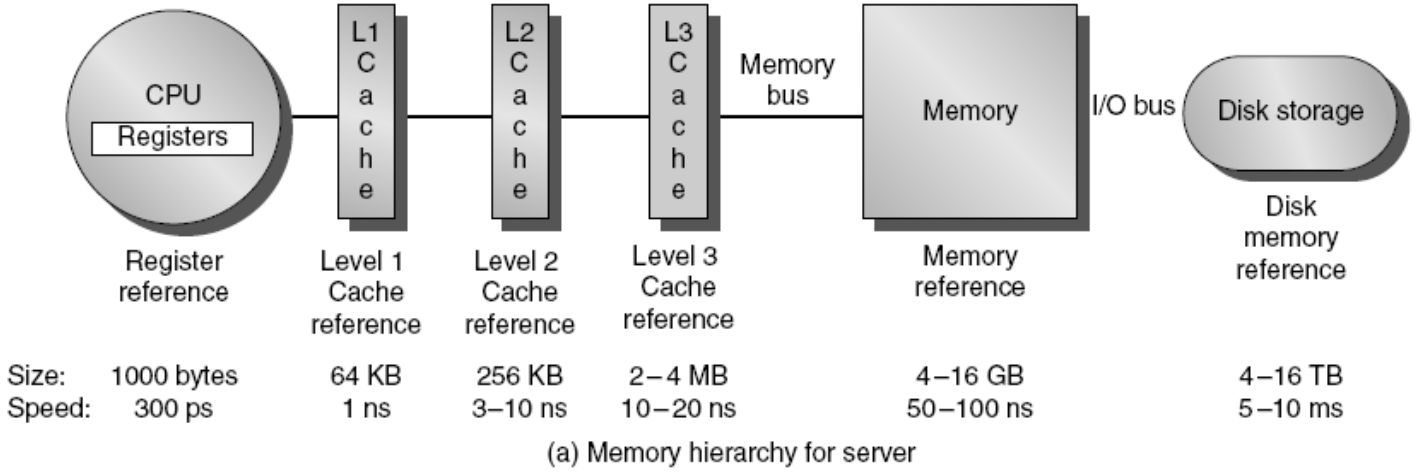


Components of every computer belong to one of these five categories

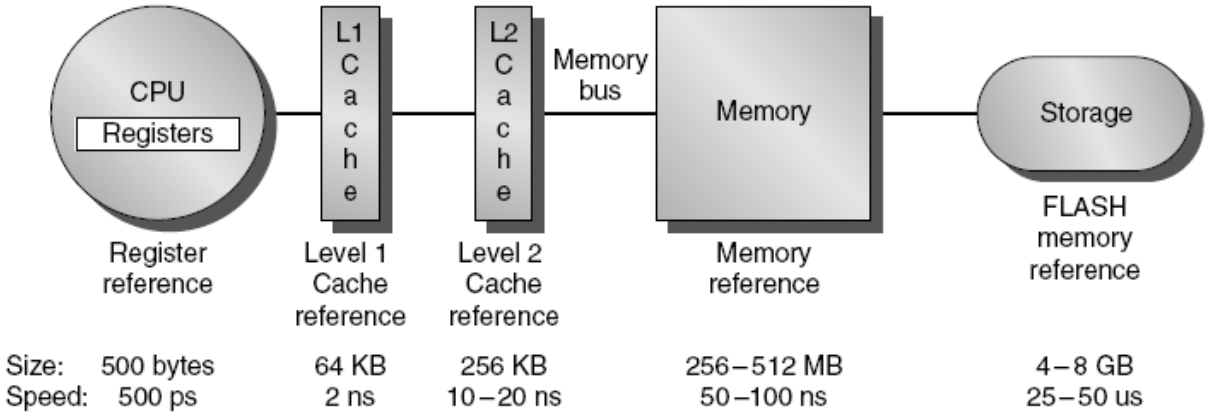
Memory Layout



Memory Hierarchy



(a) Memory hierarchy for server

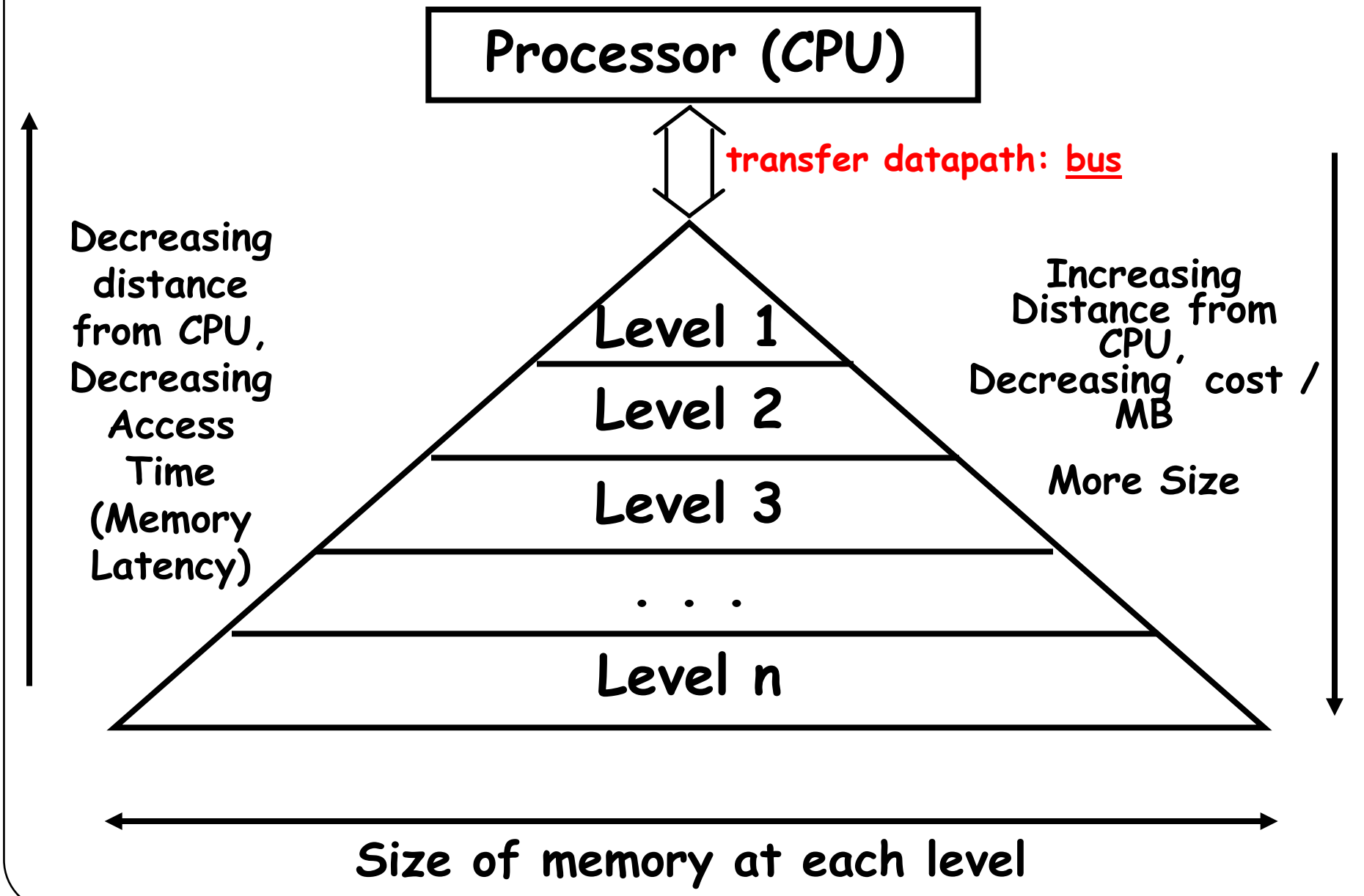


(b) Memory hierarchy for a personal mobile device

Memory Technology in Hierarchy

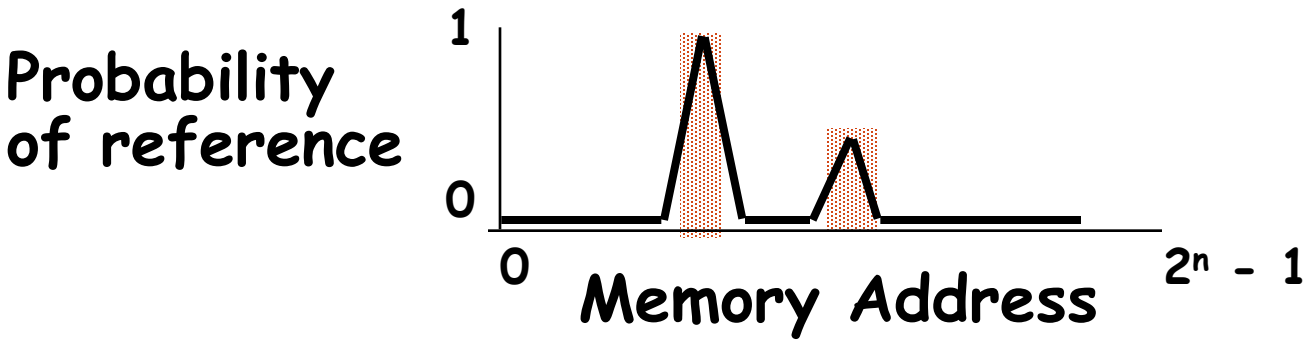
- Random Access:
  - "Random" is good: access time is the same for all locations
  - DRAM: Dynamic Random Access Memory
    - High density, low power, cheap, slow
  - SRAM: Static Random Access Memory
    - Low density, high power, expensive, fast
- "Not-so-random" Access Technology:
  - Access time varies from location to location and from time to time
  - Examples: Disk, CDROM
- Sequential Access Technology: access time linear in location (e.g.,Tape)
- We will concentrate on random access technology
  - The Main Memory: DRAMs + Caches: SRAMs

Memory Hierarchy Pyramid



Why Hierarchy is implemented:

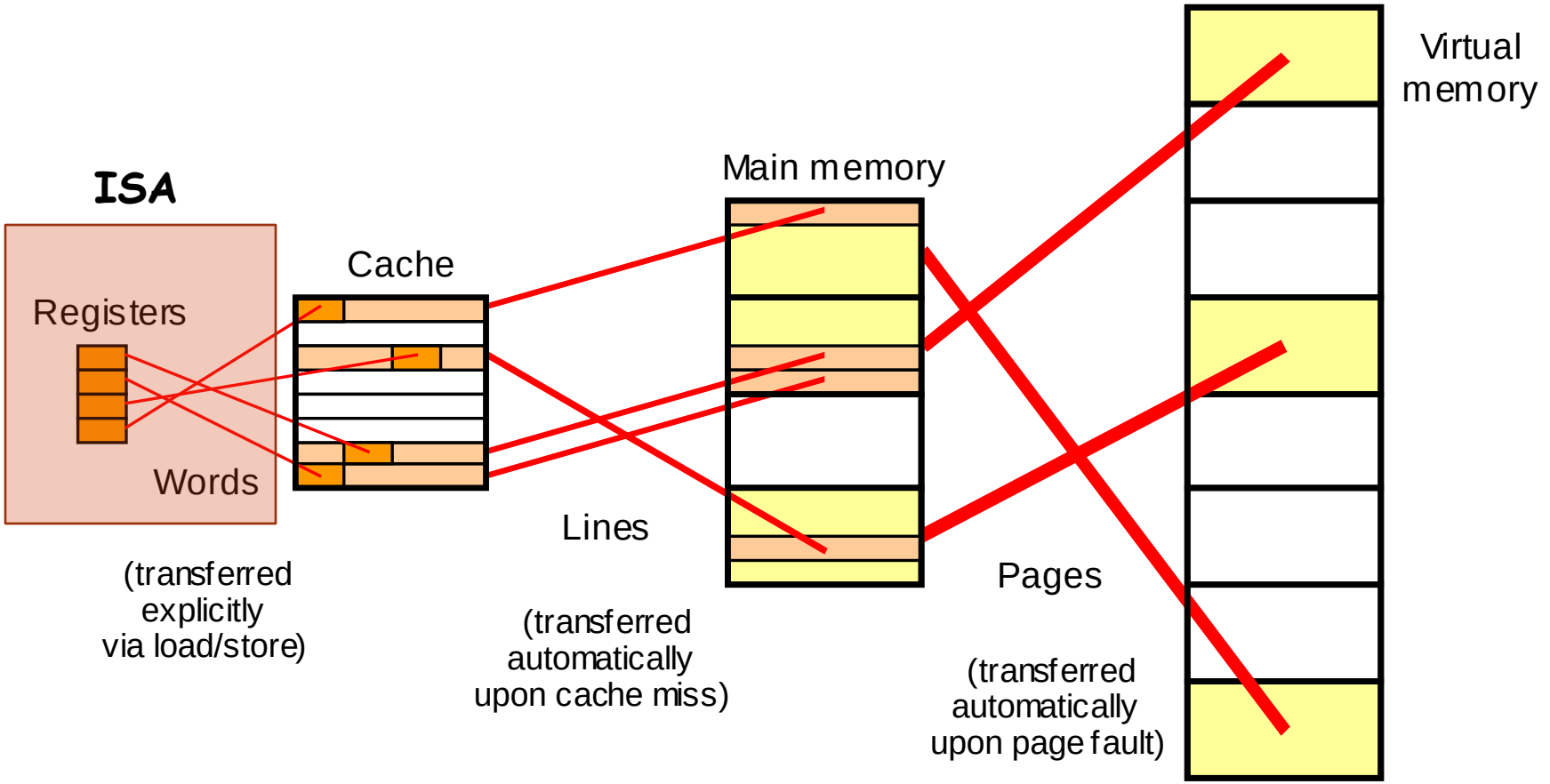
- The Principle of Locality:
  - Programs access a relatively small portion of the address space at any second



° **Temporal Locality (Locality in Time):⇒ Recently accessed data tend to be referenced again soon**

° **Spatial Locality (Locality in Space):⇒ nearby items will tend to be referenced soon**

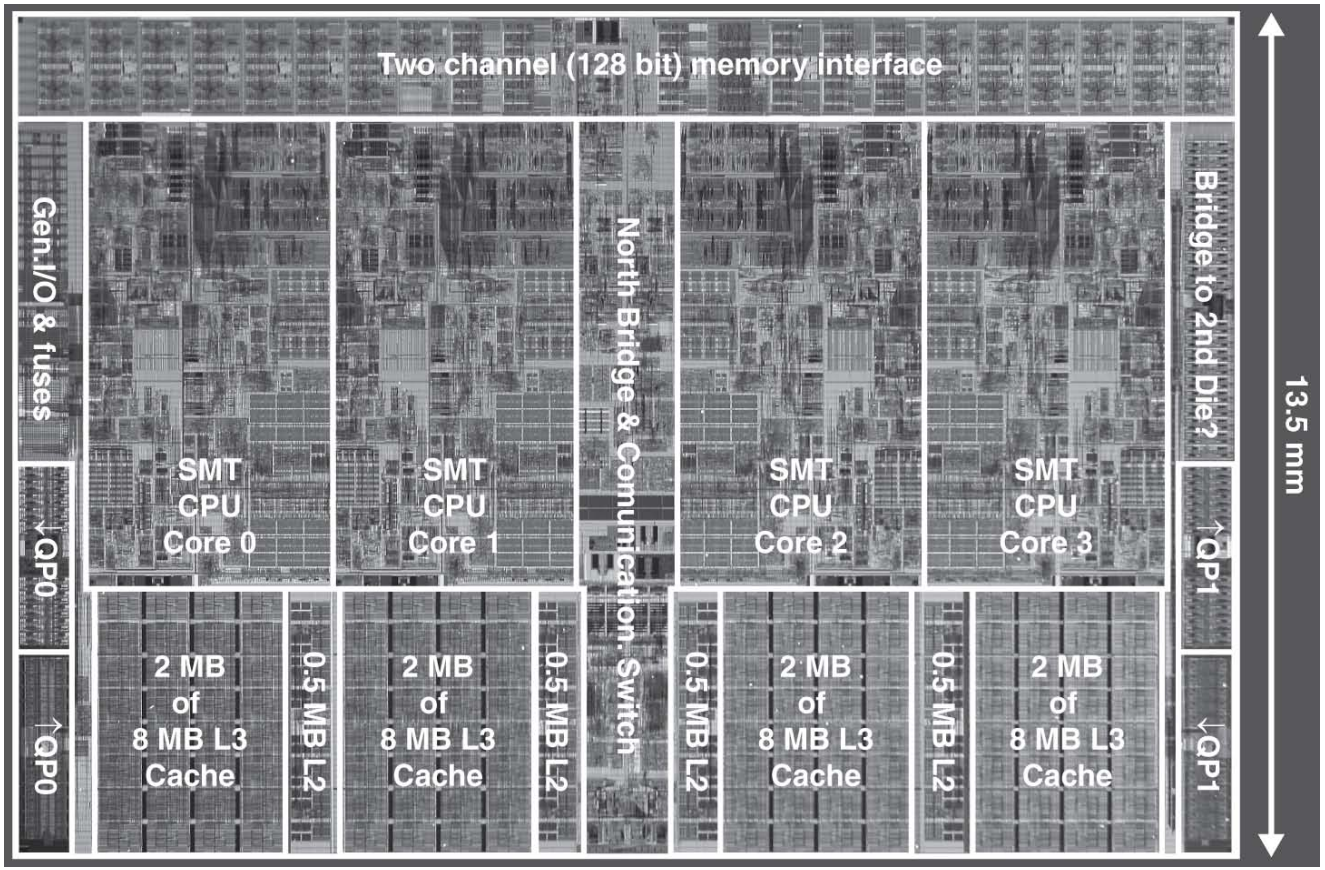
Memory Hierarchy: The Big Picture



Data movement in a memory hierarchy.

Multilevel On-Chip Caches

Intel Nehalem 4-core processor

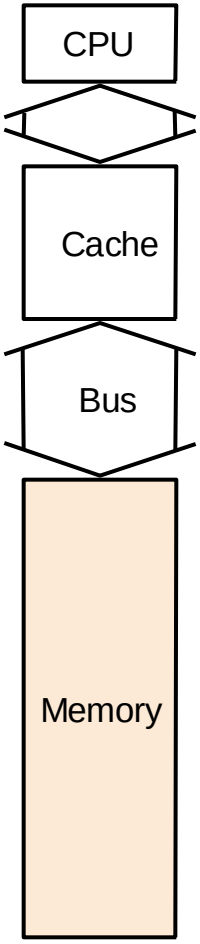


Per core: 32KB L1 I-cache, 32KB L1 D-cache, 512KB L2 cache, 8 MB L3 cache, 4-way interleaved, shared by all cores.

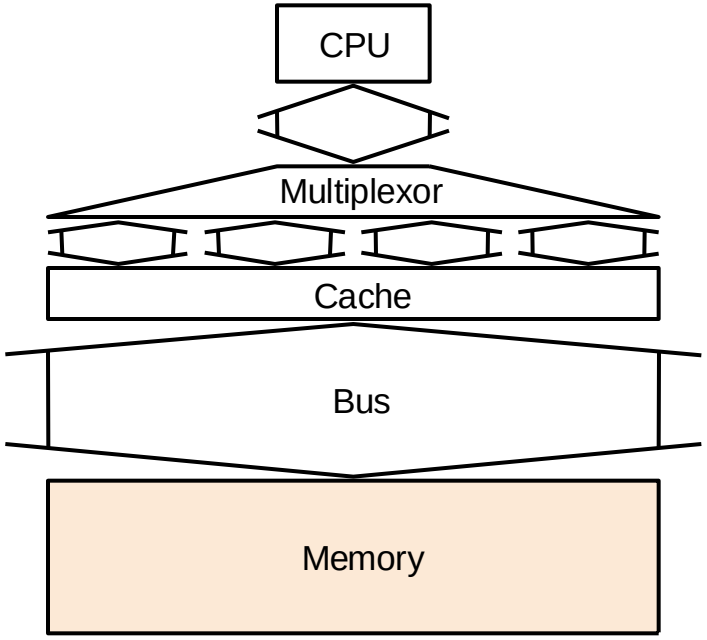
Cache ⇔ Main Memory

- Random Access Memory (vs. Serial Access Memory)
- Cache uses *SRAM*: Static Random Access Memory
  - No refresh (6 transistors/bit vs. 1 transistor
- Main Memory is *DRAM*: Dynamic Random Access Memory
  - Dynamic since needs to be refreshed periodically
  - Addresses divided into 2 halves (Memory as a 2D matrix):
    - *RAS* or *Row Access Strobe*
    - *CAS* or *Column Access Strobe*
- Synchronous DRAM (SDRAM): Ability to transfer a burst of data given a starting address and a burst length – suitable for transferring a block of data from main memory to cache.

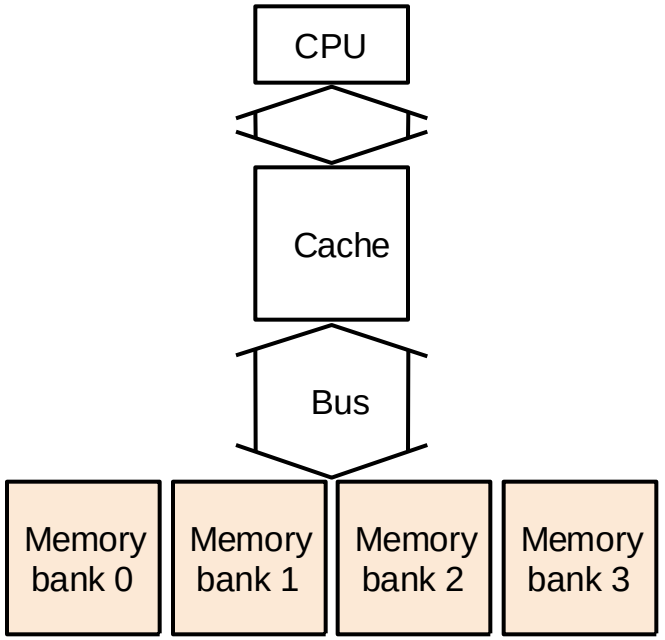
Main Memory Organizations



one-word wide
memory organization



wide memory organization
vs.
Multichannel memory org
DDR2, DDR3, DDR4,..



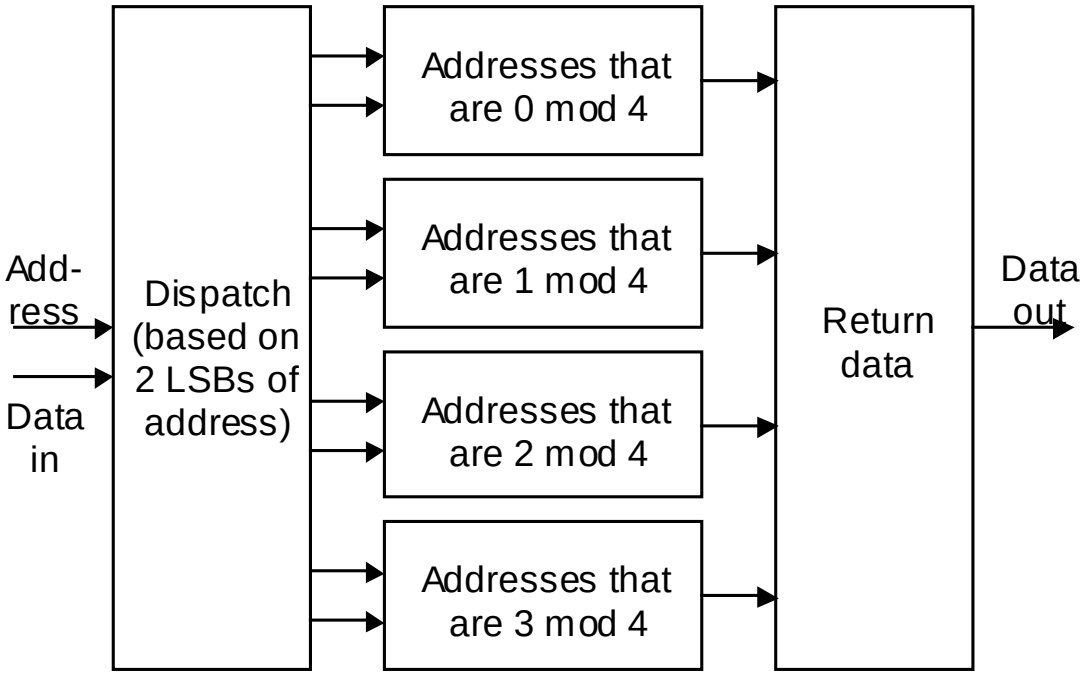
interleaved
memory organization

DRAM access time >> bus transfer time

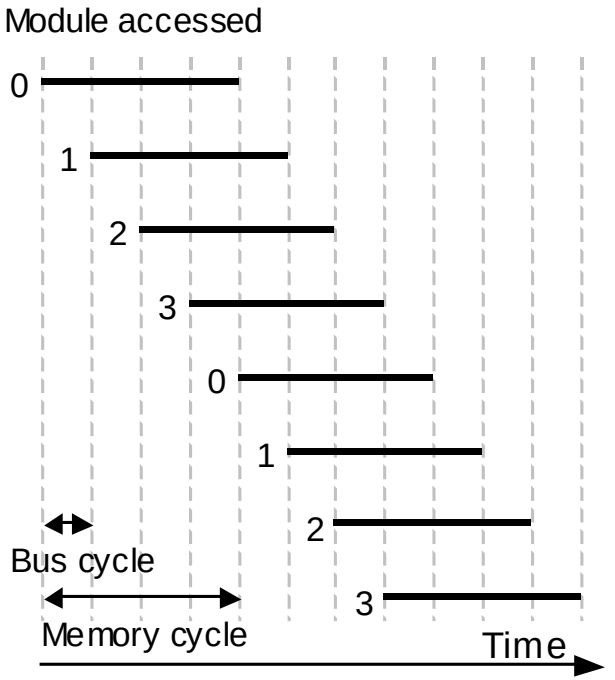
DDR= Double data rate
DIMM= Dual inline Memory module

Memory Interleaving

4 Bytes addressed = One word (contagious)



Address and Data Pipelining

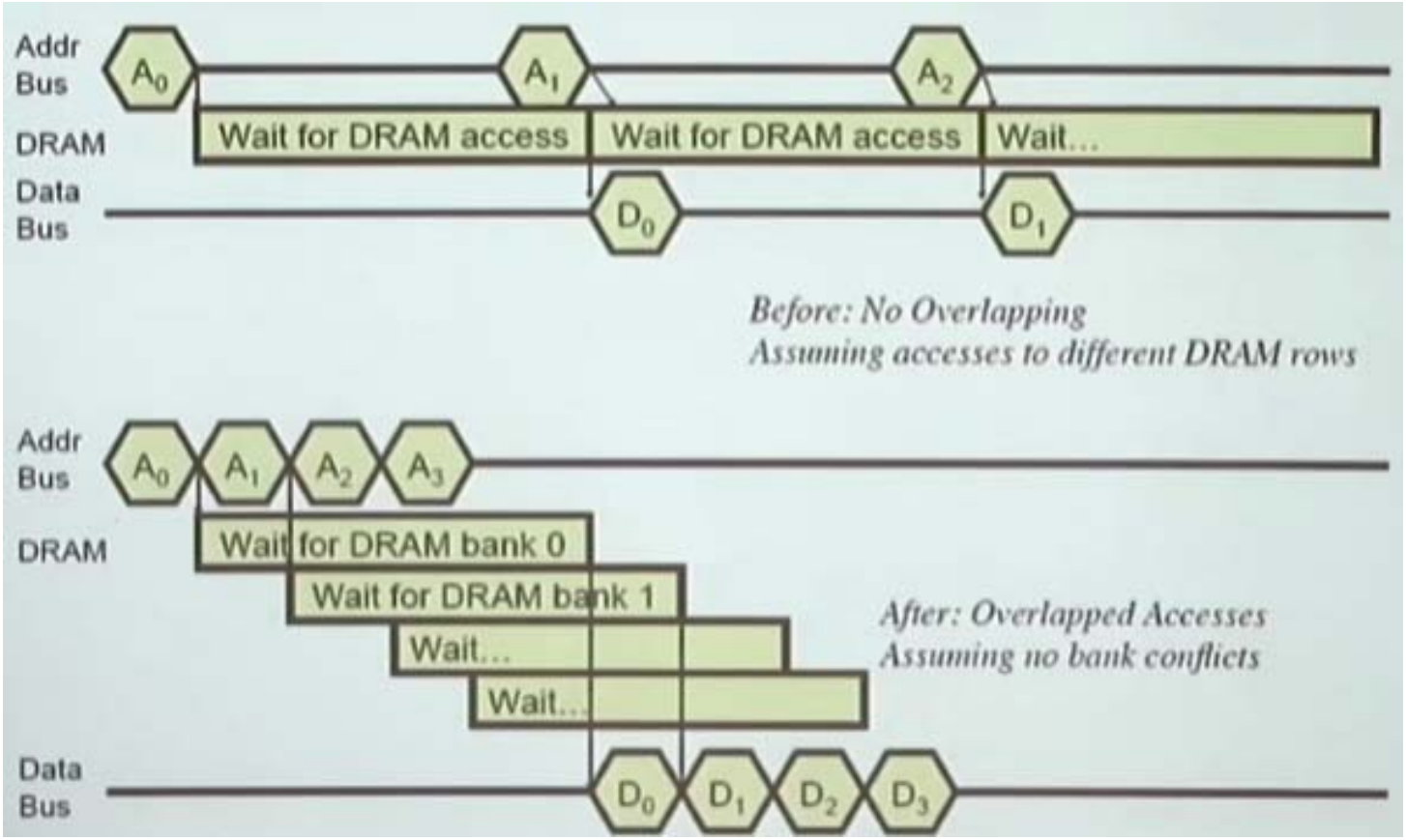


DRAM access time >> bus transfer time

Interleaved memory is more flexible than wide-access memory in that it can handle multiple independent accesses at once.

Memory Interleaving

Address and Data Pipelining



DRAM access time >> bus transfer time

Virtual Memory

- Idea 1: Many Programs sharing DRAM Memory so that context switches can occur
- Idea 2: Allow program to be written without memory constraints – program can exceed the size of the main memory
- Idea 3: Relocation: Parts of the program can be placed at different locations in the memory instead of a big chunk.
- Virtual Memory:

  (1) DRAM Memory holds many programs running at same time (processes)

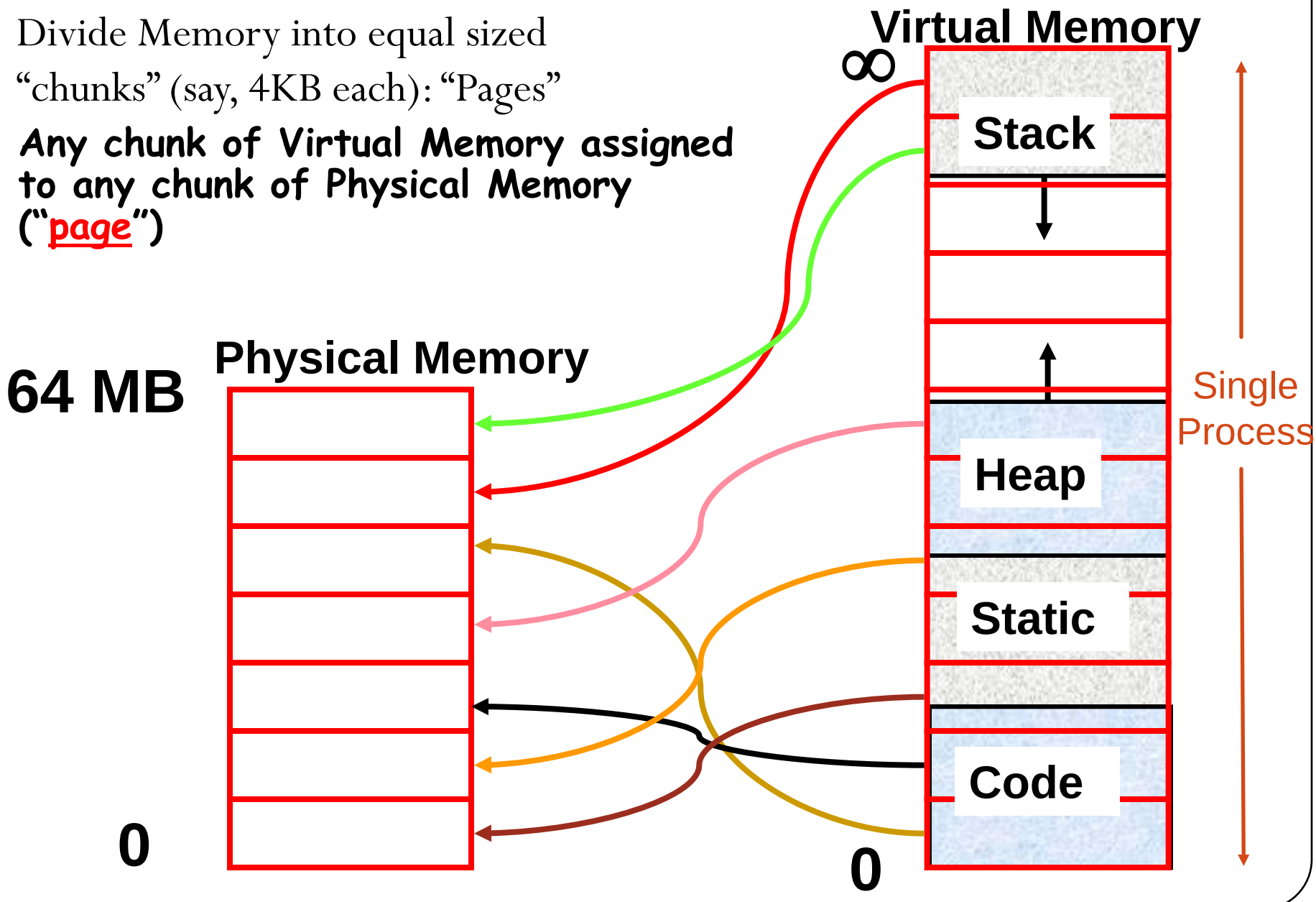
  (2) use DRAM Memory as a kind of "cache" for disk

Virtual Memory Terminology

- Each process has its own private "virtual address space" (e.g., 2^{32} Bytes); CPU actually generates "virtual addresses"
- Each computer has a "physical address space" (e.g., 128 MegaBytes DRAM); also called "real memory"
- Address translation: mapping virtual addresses to physical addresses
  - Allows multiple programs to use (different chunks of physical) memory at same time
  - Also allows some chunks of virtual memory to be represented on disk, not in main memory (to exploit memory hierarchy)

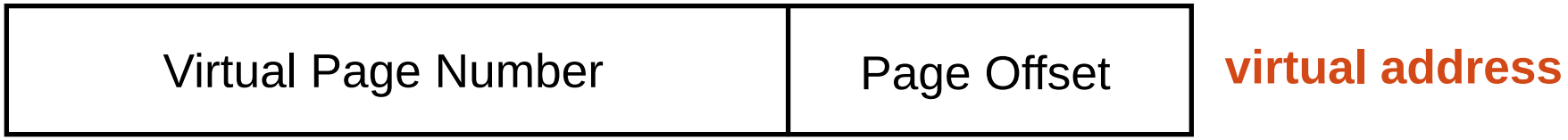
Mapping Virtual Memory to Physical Memory

- Divide Memory into equal sized "chunks" (say, 4KB each): "Pages"
- **Any chunk of Virtual Memory assigned to any chunk of Physical Memory ("page")**



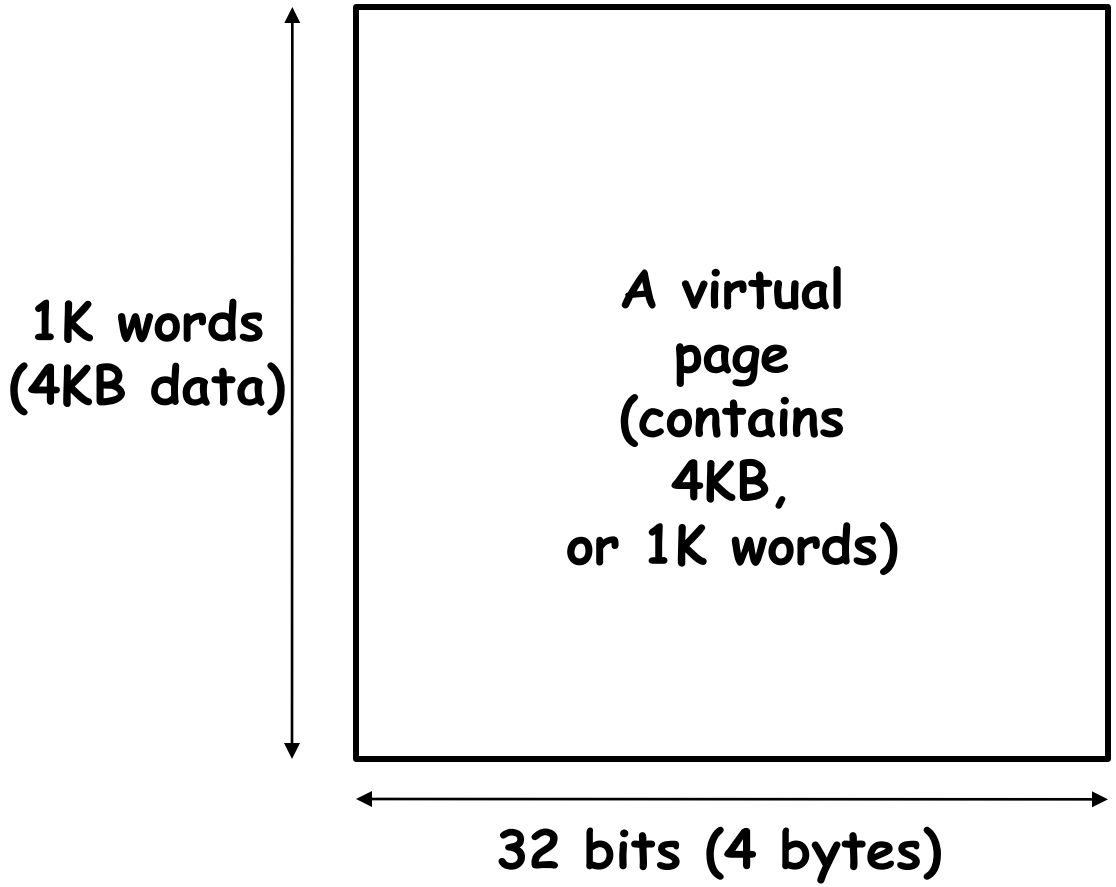
How to Perform Address Translation?

- VM divides memory into equal sized pages (4KB-16KB)
- Address translation relocates entire pages
  - if make page size a power of two, the virtual address separates into two fields:
  - like cache index, offset fields
  - offsets within the pages do not change

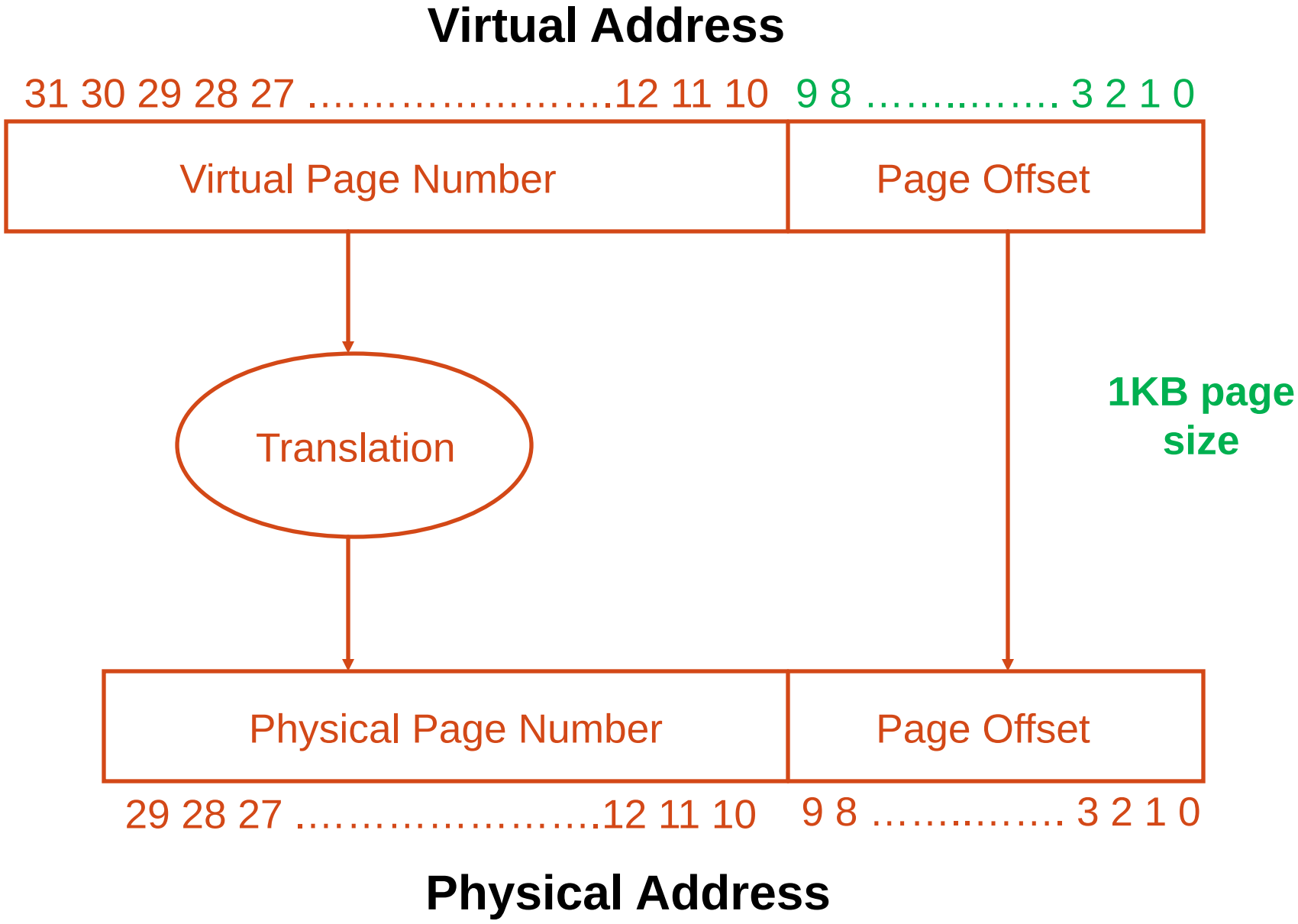


32-bit Virtual Address (4 KB Page)

20-bit virtual page number | 10-b word number within page | 2-b byte offset



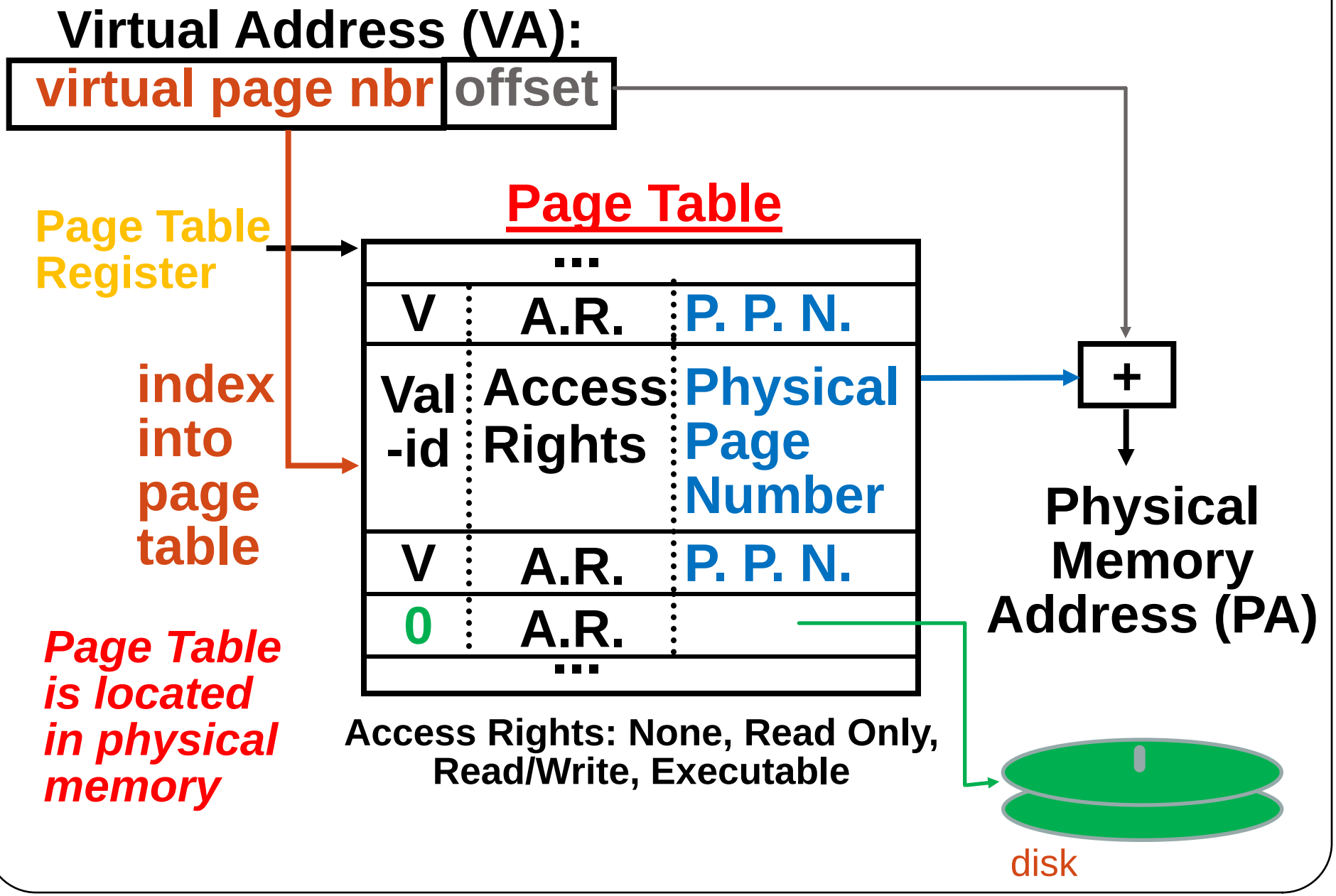
Mapping Virtual to Physical Address



Address Translation

- A page table is a data structure in DRAM main memory which contains the mapping of virtual pages to physical pages
  - There are several different ways, all up to the operating system, to keep this data around
- Each process running in the system has its own page table

Address Translation: Page Table

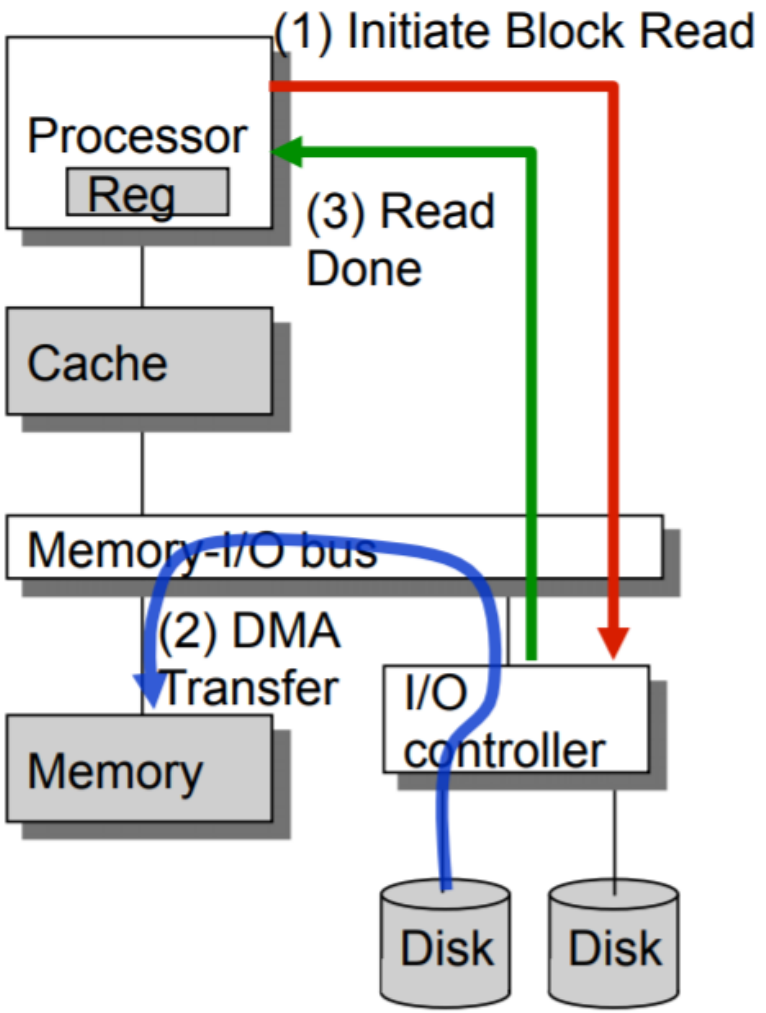


Handling Page Faults

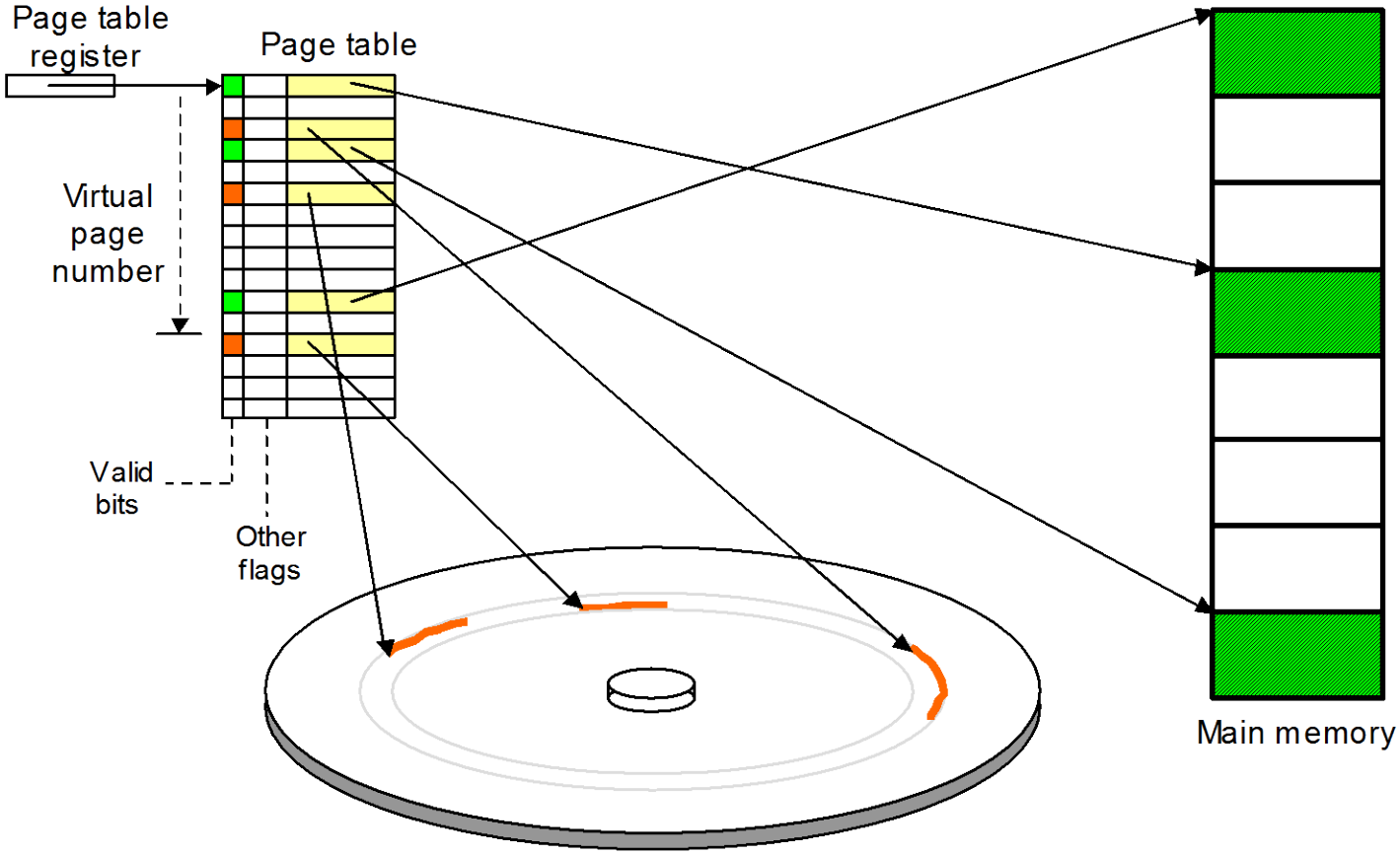
- A page fault is like a cache miss
  - Must find page in lower level of hierarchy
- If valid bit in page table is zero, the Physical Page Number points to a page on disk
- When OS starts new process, it creates space on disk for all the pages of the process, sets all valid bits in page table to zero, and all Physical Page Numbers to point to disk
  - called Demand Paging - pages of the process are loaded from disk only as needed

Servicing a Page Fault

- (1) Processor signals controller
  - Read block of length P starting at disk address X and store starting at memory address Y
- (2) Read occurs
  - Direct Memory Access (DMA)
  - Under control of I/O controller
- (3) Controller signals completion
  - Interrupt processor
  - OS resumes suspended process

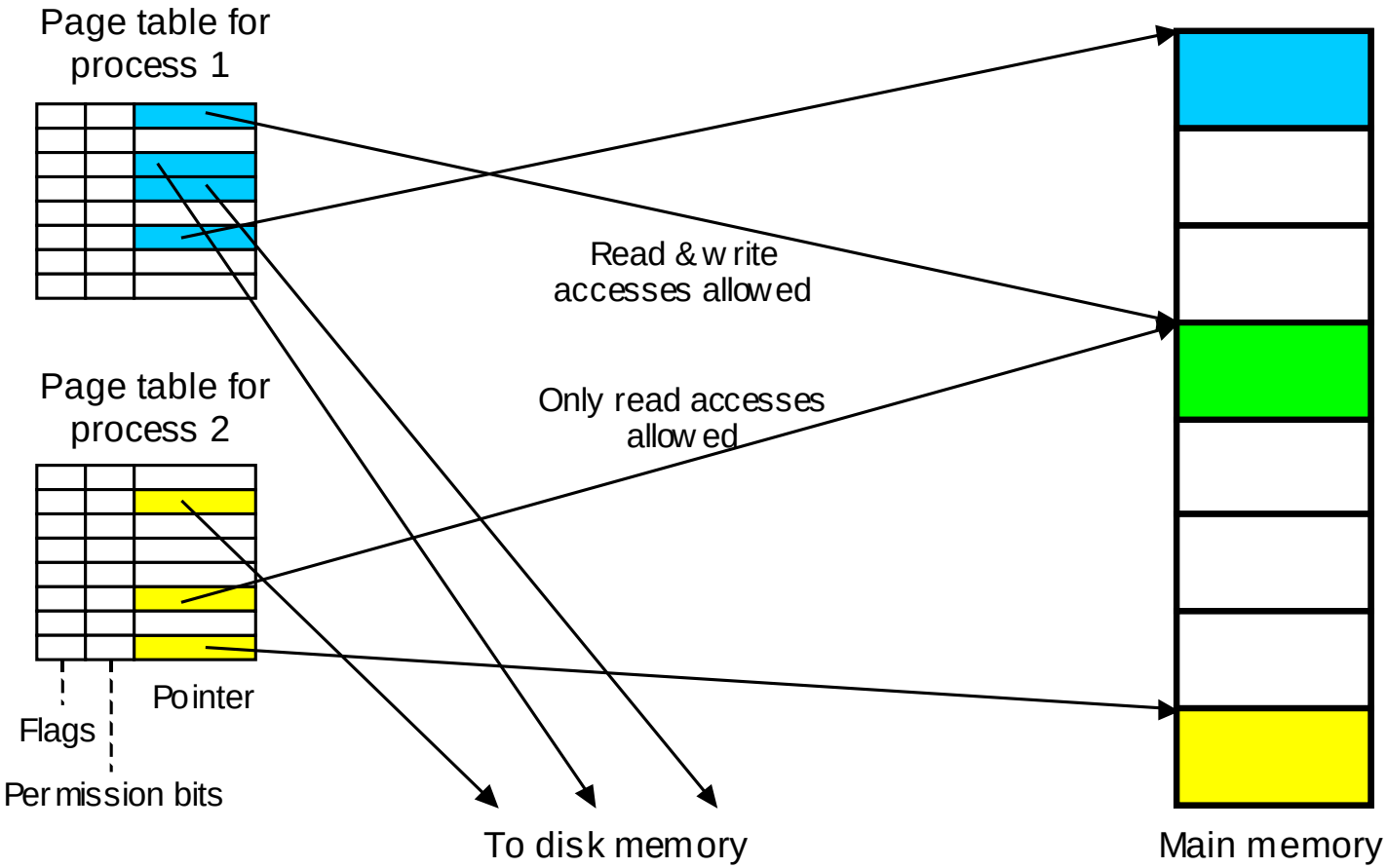


Page Tables and Address Translation



The role of page table in the virtual-to-physical address translation process.

Protection and Sharing in Virtual Memory



Virtual memory as a facilitator of sharing and memory protection.

How Translate Fast?

- Problem: Virtual Memory requires two memory accesses!
  - one to translate Virtual Address into Physical Address (page table lookup)
  - one to transfer the actual data (cache hit)
  - But Page Table is in physical memory! => 2 main memory accesses!
- Observation: since there is locality in pages of data, must be locality in virtual addresses of those pages!
- Why not create a cache of virtual to physical address translations to make translation fast? (smaller is faster)
- For historical reasons, such a "page table cache" is called a Translation Lookaside Buffer, or TLB

Typical Translation Look Aside Buffer (TLB) Format

Virtual Page No	Physical Page No	Valid	Ref	Dirty	Access Rights

"tag" "data"

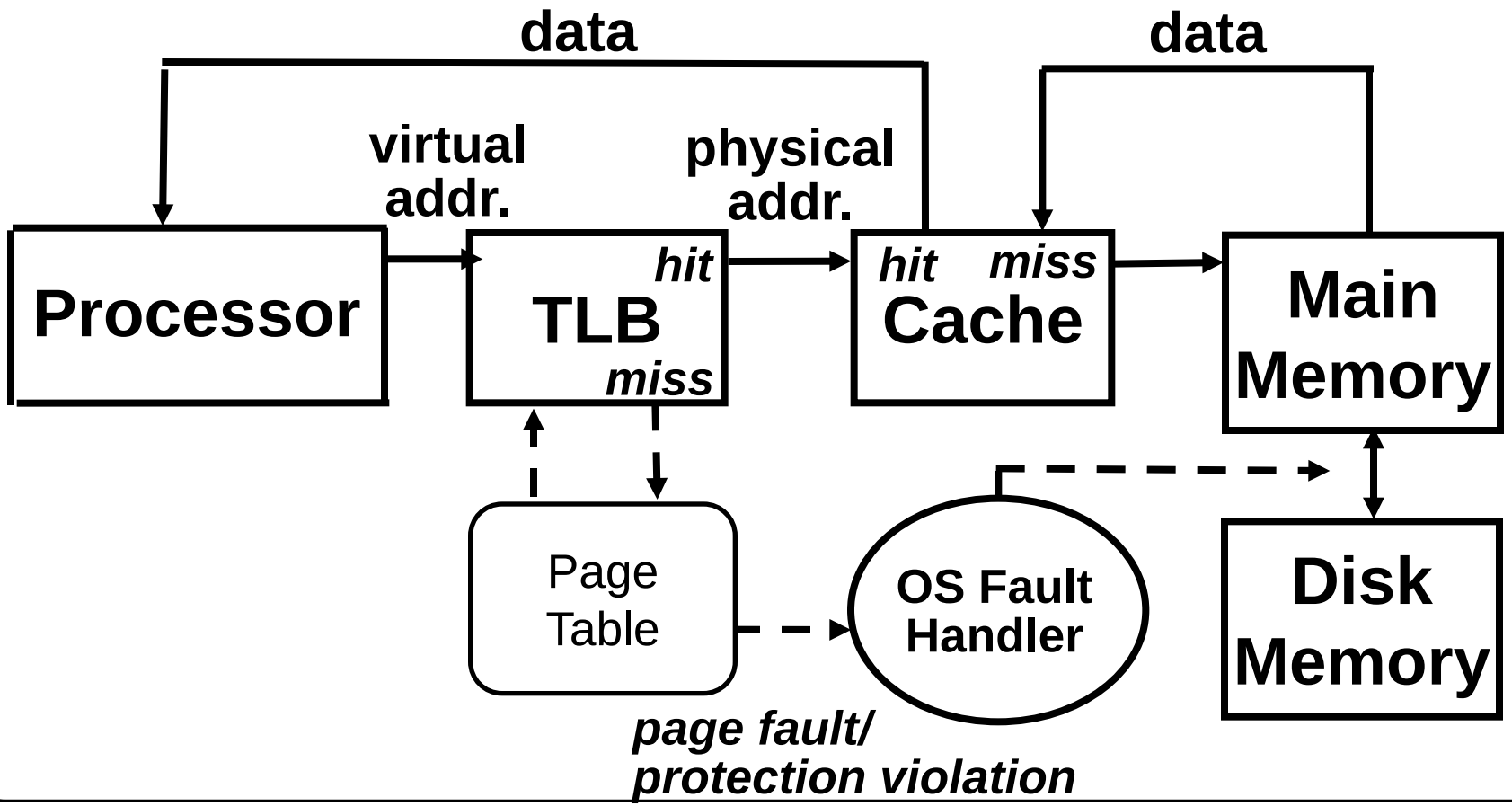
- TLB just a cache of the page table mappings

- Dirty: since use write back, need to know whether or not to write page to disk when replaced
- Ref: Used to calculate LRU on replacement

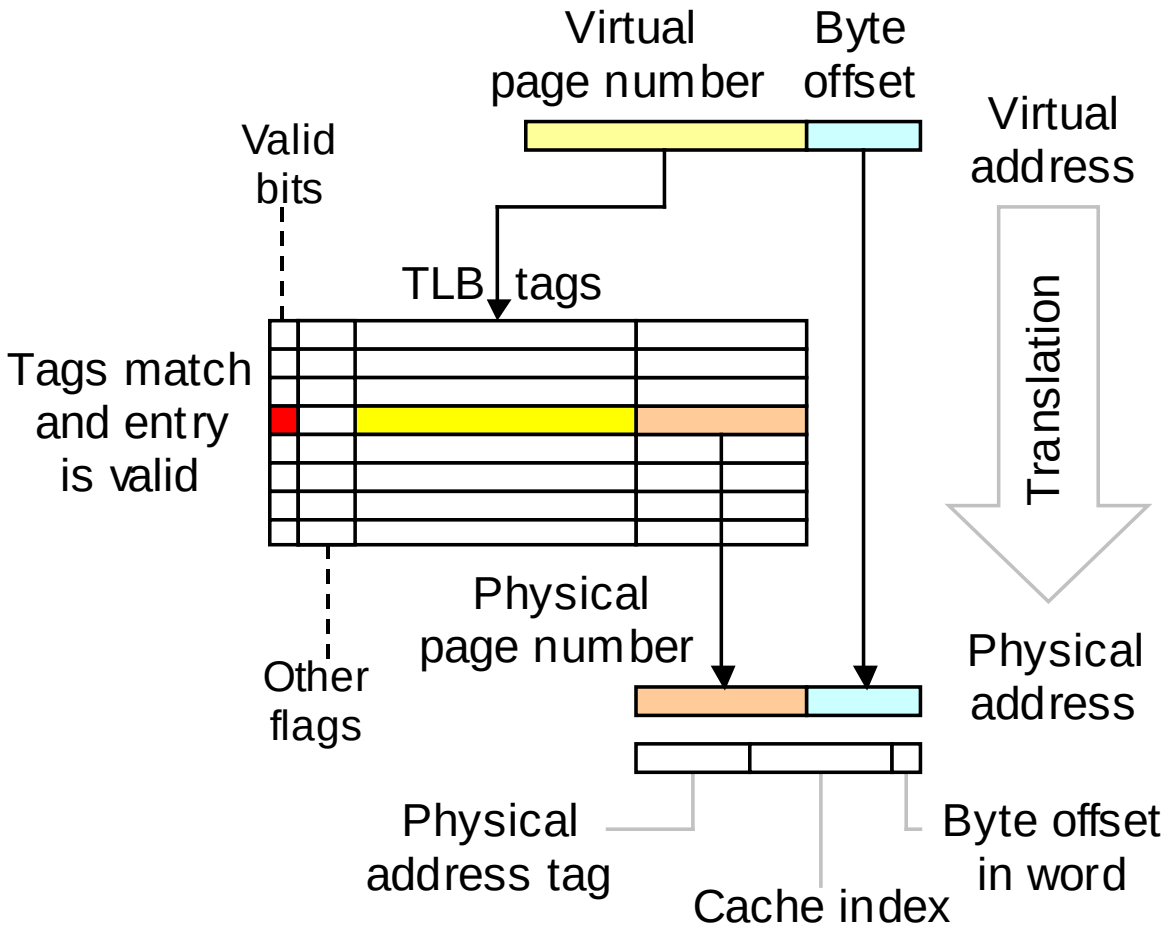
- TLB access time comparable to cache (much less than main memory access time)

Translation Look-Aside Buffers

- **TLB is usually small, typically 32-4,096 entries**
- **Like any other cache, the TLB can be fully associative, set associative, or direct mapped**

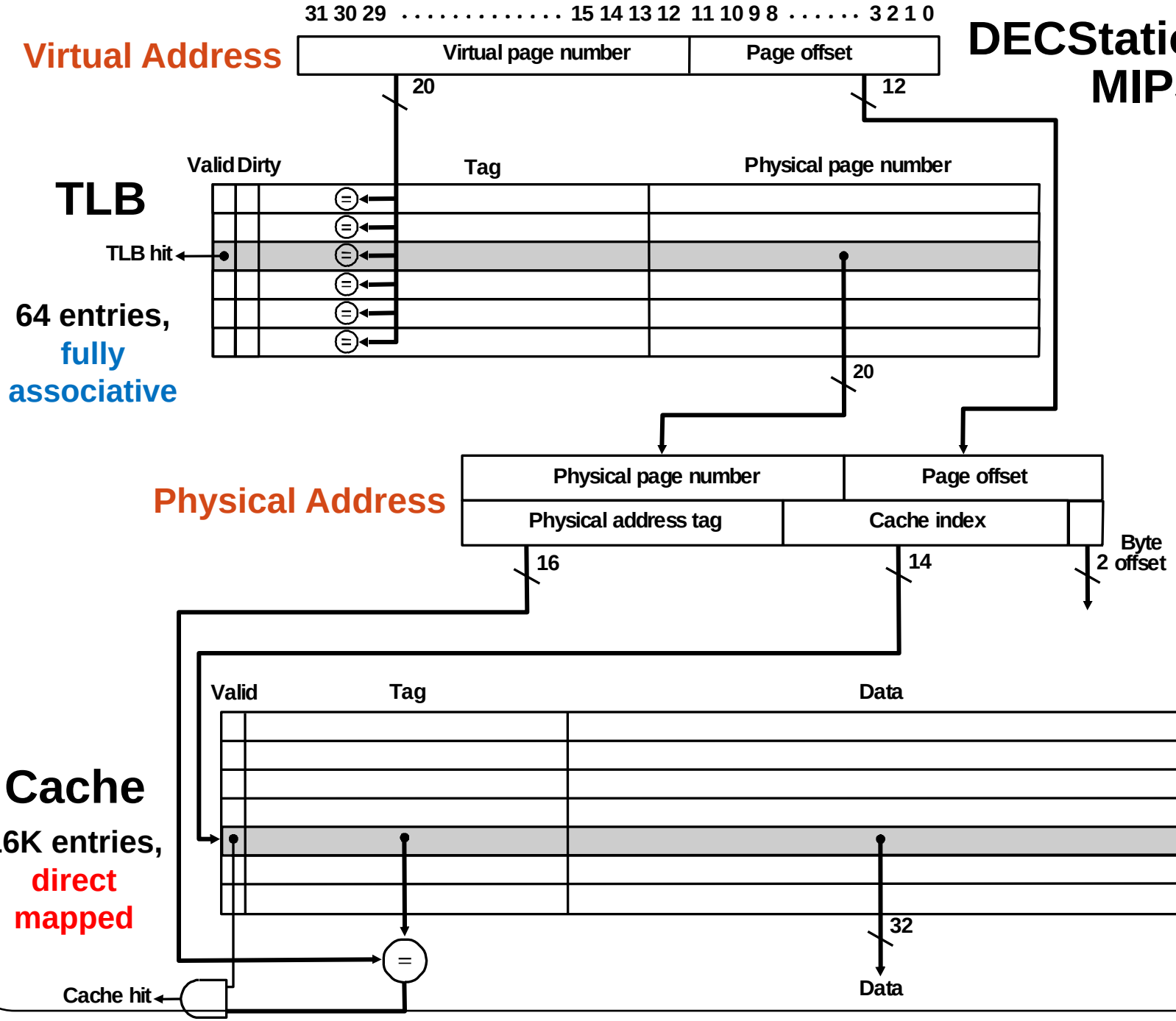


Translation Lookaside Buffer

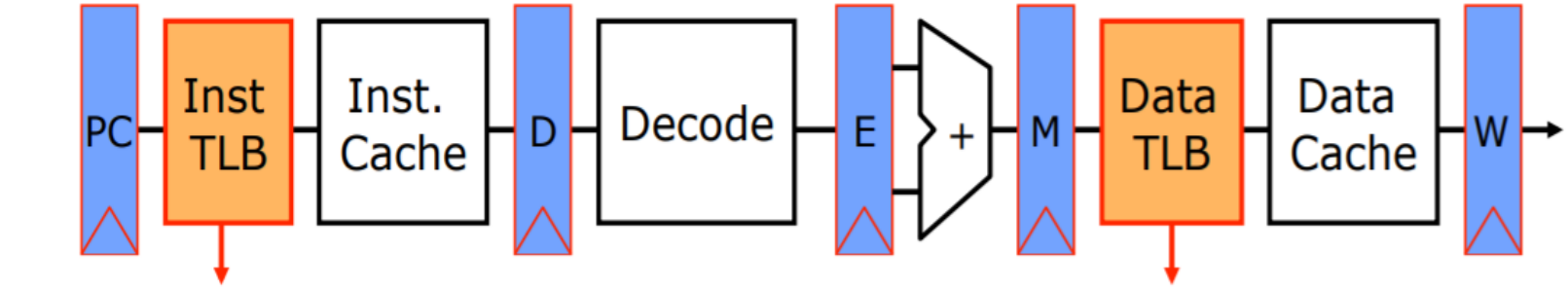


Virtual-to-physical address translation by a TLB and how the resulting physical address is used to access the cache memory.

DECStation 3100/ MIPS R2000



Address Translation in CPU Pipeline



Optimizing for Space

- Page Table too big!
  - 4GB Virtual Address Space ÷ 4 KB page
    $\Rightarrow 2^{20}$ (~ 1 million) Page Table Entries
    $\Rightarrow$ 4 MB just for Page Table of single process!
- Variety of solutions to tradeoff Page Table size for slower performance when miss occurs in TLB

  Use a limit register to restrict page table size, Multilevel page table, Paging page tables, etc.

  (Take O/S Class to learn more)

Comparing the 2 levels of hierarchy

Cache	*Virtual Memory*
Block or Line	Page
Miss	Page Fault
Block Size: 32-64B	Page Size: 4K-16KB
Placement: Direct Mapped, N-way Set Associative	Fully Associative
Replacement: LRU or Random	Least Recently Used (LRU) approximation
Write Thru or Back	Write Back
How Managed: Hardware	(Operating System)

Book Ref

Topics:

1. Virtual Machines and Virtual Memory: Sections 5.6-5.7
2. Cache Memory: Sections Sections 5.1-5.5

Assignment I Due on Sat 17 Apr 1500